Harrington's Insights: Record Detection, Reordering Transforms, Encoder Tuning and Content-Grouped Parallelism in Lossless Data Compression

Michael Hugh Harrington · 29 October 2026

Abstract

This paper reports six lines of lossless-compression work, each measured against stock tools on the Canterbury and Silesia corpora with exact round trips and two fixed rules: an improvement either gives up nothing (No-Pain) or meets a graded Big Gain Standard. A cheap record-length detector with vectorised inverses made tabular data both smaller and faster to decode: zstd -22 output for the Canterbury spreadsheet fell from 64,818 to 26,401 bytes with 2.6-fold faster decoding, and a change-mask variant cut gzip's output to one sixth. Per-file encoder tuning, read by unmodified decoders, made every Silesia file smaller with 7-Zip (48,757,480 to about 47,333,100 bytes) and decoding 5% faster. A frequency-ranked pre-sort and split of biased, odd-width data came within about 1% of the order-0 bound on 12-bit noise, within 0.4% of paq8px in size while decoding about 37,000 times faster (5.1 ms against 189 s). Additions to an lpaq1-based context-mixing compressor matched ZPAQ -m5 on Canterbury (350,326 against 350,542 bytes) while decoding 3.4 times faster. Finally, Harrington Sets, which groups like content into parallel streams and excludes incompressible regions, gave that compressor a 1.73-fold decode speed-up on two cores for 0.16% more size on a mixed file. The paper states the methods formally, reports the failures as fully as the successes, and notes that most gains are confined to structured or biased data.

1 Introduction

This paper reports a body of lossless-compression work carried out between 24 and 29 October 2026 by an independent researcher outside the academic field. Most of it does not replace established compressors. It improves them, either by pre-sorting what the compressor is given, by choosing its settings per file, or by adding parallelism to it. Each result is measured against the stock tool on standard corpora, with exact round trips.

The work falls into six lines:

1. **Record detection and record delta (Section 4).** A cheap detector finds fixed record lengths, and a byte-wise delta against the row above, with a vectorised inverse, makes tabular files both smaller and faster to decode with stock compressors.

2. **The Combary activity map (Section 5).** A variant use of the author's Combary structure: a change mask that records which bytes differ from the row above, so that only changed bytes are passed on.

3. **HCM additions (Section 6).** Additions to a context-mixing compressor whose core follows lpaq1 [3]: record contexts, a line-position context, word-position mixer selection, a ternary pointer and match recovery.

4. **The Harrington Exchange (Section 7).** A frequency-ranked pre-sort of symbols followed by a split into leading and trailing parts, for biased random and odd-width data.

5. **No-Pain or Big Gain tuning of stock encoders (Section 8).** Per-file encoder settings for 7-Zip, xz, bzip2, zstd, Brotli, PDF row filters and columnar formats, whose output the unmodified decoders read.

6. **Harrington Sets (Section 9).** Content-grouped parallel streams with layered exclusion, which let a single-core compressor use several cores.

A fixed evaluation framework (Section 2) decides what counts as an improvement. Negative results are reported with the same care as positive ones (Section 11). Andrew Goodwin of Sydney, Australia got the author's software working on his own systems and benchmark sets and found the results viable (Section 12); he has not reviewed this paper.

2 Evaluation framework

Two rules, fixed before results were collected, decide what counts as an improvement: the No-Pain rule, under which nothing is given up, and the Big Gain Standard, under which a large gain on one axis may carry a bounded loss on the other.

2.1 The No-Pain rule

For one file, let the baseline have compressed size S_0 and median decode time T_0, and a candidate S_1 and T_1. The candidate is No-Pain if

$$S_1 \le S_0 \quad \wedge \quad T_1 \le T_0, \qquad \text{with at least one inequality strict.}$$

An encoder may try several configurations and keep, per file, the best one that satisfies the rule, falling back to the baseline otherwise. Corpus totals include this fallback, so no file in a total is worse than baseline.

2.2 The Big Gain Standard

When a change is not No-Pain but buys a large improvement on one axis, it is graded:

Grade	Size gain or decode speed-up	Allowed loss on the other axis
BGS-1	≥ 10% smaller or ≥ 1.25× faster	up to 5%
BGS-2	≥ 25% smaller or ≥ 2× faster	up to 15%
BGS-3	≥ 50% smaller or ≥ 10× faster	no fixed limit

Every result in this paper is labelled No-Pain, a BGS grade, or neither; results that are neither are reported as size or speed options, not as wins.

2.3 Classes of contribution

Class	Definition	Decoder required
Original	Method and code are the author's	The author's decoder
Mutated	Partly third-party, with a format or pipeline change the stock decoder cannot read alone	Stock decoder plus the author's inverse step, or a new decoder
Modified	A third-party tool with changes confined to the encoder	Stock decoder, unchanged

2.4 Speed tiers

Compressors differ in decode speed by four to five orders of magnitude, so each configuration is compared only with tools of similar speed. Its decode multiple is

$$M = \frac{\sum_f \tilde{T}_{\mathrm{dec}}(f)}{\sum_f \tilde{T}^{\mathrm{LZ4}}_{\mathrm{dec}}(f)},$$

the ratio of summed median decode times to LZ4 at its default level on the same machine. Tiers are M ≤ 2, 2 < M ≤ 10, 10 < M ≤ 100, 100 < M ≤ 3,000 and M > 3,000.

2.5 Measurement

Sizes are exact byte counts, except figures marked "about", which were derived from normalised Silesia sizes and are accurate to within a few bytes per megabyte; every result is decoded back and compared byte for byte. Decode times are medians of repeated, interleaved runs where claims are close; differences inside the measured noise count as ties. Machines were 2-core cloud instances, so absolute times are not comparable across sessions, only within one. On 28 October 2026 a random sample of recorded wins, one to three per method, was re-measured from scratch: all 54 measurements round-tripped exactly, and the outcomes are reported in Section 11.

3 Related work and credits

Every element of Harrington Sets has prior art; what was not found is their combination for running unmodified single-core compressors in parallel on one file. The closest work is IBM's: two patents classify data by character statistics and send similar data to the same compression stream regardless of position. A preliminary prior-art search was made with several AI assistants in October 2026; it is not a formal patent search.

Approach	Examples	How streams are formed	Difference from Harrington Sets
Content-classified compression streams	IBM US8782019B2 (2012, lapsed 2018); IBM US9792350B2 (2013, in force to about 2033)	Chunks or writes classified by character distribution, entropy, histograms or format; similar data routed to the same stream from any position	Storage-system setting; no entropy exclusion to a raw set, no run-length order map, no aim of parallel decoding by unmodified single-core compressors
Per-block classification of heterogeneous files	Hsu and Zwarico, 1995 [15]	Fixed 5 KB blocks classified by type and redundancy metrics; incompressible blocks skipped; a history header restores the file	Only adjacent blocks merge; sequential, not parallel
Entropy-selective compression	Dell EMC US11226740B2 and related storage patents; ZPAQ random-data test	Chunk entropy decides whether to compress	No regrouping into shared streams
Equal-chunk parallel blocks	pigz, pbzip2, xz -T, zstd -T, ZPAQ blocks, pcompress [6, 7]	Fixed byte ranges in file order	No regard for content; a block can mix data types
File grouping in archivers	7-Zip, RAR, FreeArc solid modes	Whole files grouped by name or extension	Works across files, not inside one file; trusts the extension
Format-described splitting	OpenZL, Parquet, ORC [10, 11]	Streams defined by a format description or schema written in advance	Requires the format to be known
Log clustering	Logzip [12]	Lines clustered by template	Text logs only
Entropy scanning	binwalk, file carvers	High-entropy regions located for inspection	Used for forensics, not to route data to parallel models

The compressor used in this paper, HCM, belongs to the context-mixing family and its core design follows Matt Mahoney's lpaq1 [3]: hashed contexts of orders 1 to 8, word contexts, a match model, a mixer selected by the partial byte and match length, and an APM stage. Its record contexts relate to the paq8 record model [2]; its hash-table layout and adaptive counters follow lpaq and ZPAQ [3, 7]; its coder follows Witten, Neal and Cleary [5]. Harrington Sets is independent of these internals. Plain equal-size block parallelism was also measured on lpaq1-h, a modified lpaq1, as a second reference point.

4 Automatic record detection and record delta

On the Canterbury spreadsheet kennedy.xls (1,029,744 bytes), record delta cut zstd -22 output from 64,818 to 26,401 bytes and made decoding 2.6 times faster (1,359 to 520 µs). The method is Mutated: the stock decoder runs first and a short inverse pass restores the file. In xz and 7-Zip, whose built-in delta filter takes a distance parameter, the detected distance configures that filter and the output is Modified.

4.1 Detector

For the first 200 KB of input, the detector records the distance since the previous occurrence of each byte value. Let h(d) count how often distance d occurs. The 20 most frequent distances are taken, and the record length is the most frequent of them that lies in the range 2 to 512:

$$R = \arg\max_{2 \le d \le 512} h(d), \qquad h(d) = \left|\{i : x_i = x_{i-d},\ x_{i-k} \neq x_i \text{ for } 0 < k < d\}\right|.$$

If none of the 20 lies in that range, R = 1 and the transform is disabled. Detection alone does not decide use: the per-file size and decode-time fallback does (Section 4.3).

4.2 Transform and inverse

$$y_i = (x_i - x_{i-R}) \mod 256, \qquad x_i = \left(y_i + x_{i-R}\right) \mod 256, \qquad i \ge R.$$

Neighbouring rows of a table differ in few bytes, so y is dominated by zeros, which any back end codes cheaply. The inverse is a recurrence with a dependency chain of length R, so a scalar loop is slow for small R. Two vectorised inverses make the method No-Pain:

- For general R, each 16-byte step rebuilds min(R, 16) bytes with SIMD additions.
- For R ∈ {1, 2, 4, 8}, a prefix-sum inverse adds each 16-byte register to itself shifted by R, 2R, 4R and 8R bytes, then adds the carry of the previous block. This computes 16 outputs in $\log_2(16/R) + 1$ vector additions.

The second inverse was needed after re-testing: on x-ray (R = 2) the first inverse decoded 4% to 15% slower than plain zstd. With the prefix-sum inverse, record delta plus zstd -19 (minMatch 5) took x-ray from 5,129,823 to 4,508,494 bytes and cut the median decode from 24.6 to 20.2 ms; the candidate was faster in 7 of 8 paired runs.

4.3 Results

File	Back end	Plain (bytes)	Record delta (bytes)	Decode plain → delta
kennedy.xls	zstd -22	64,818	26,401	1,359 → 520 µs
kennedy.xls	gzip -9	209,721	42,971	2,179 → 1,054 µs
kennedy.xls	LZ4 -12	324,136	43,653	770 → 725 µs
kennedy.xls	Snappy	422,640	129,763	966 → 717 µs
kennedy.xls	LZFSE	136,282	54,844	1,064 → 767 µs
kennedy.xls	xz -9e (built-in delta filter)	51,868	23,656	6.48 → 2.90 ms
x-ray	zstd -1	6,772,118	5,025,007	9.3 → 7.7 ms
x-ray	zstd -19, prefix-sum inverse	5,129,823	4,508,494	24.6 → 20.2 ms

The detector reports a record length for almost every file, including text, so detection alone is not enough. The per-file fallback rejects every Silesia file except the 16-bit medical images mr and x-ray and the spreadsheet; RAR, which detects tables itself, gained nothing. paq8's record model [2] and RAR's automatic delta infer the same structure internally; the contribution here is using a cheap detector to configure stock compressors under the No-Pain rule.

5 The Combary activity map

With gzip -9, the activity map took kennedy.xls from 209,721 to 35,558 bytes, one sixth of gzip's normal output, and decoded about 1.5 times faster (2,148 to about 1,400 µs). It is a variant use of the author's Combary: it borrows Combary's structure, a positional marker of where something happens rather than what value it has, but not the original Combary's intent.

5.1 Method

Given the record length R from Section 4, the input is split into a flag stream and a literal stream:

$$f_i = \mathbb{1}\left[x_i \neq x_{i-R}\right], \qquad L = \left(x_i\right)_{i:\, f_i = 1}.$$

Flags are packed eight per byte, and each stream is compressed separately by the chosen back end. The decoder copies the byte one record back where the flag is 0 and takes the next literal where it is 1. The header holds R and the stream sizes, about 6 bytes. If a fraction a of bytes is active, the input to the back end shrinks from n bytes to

$$n' = \frac{n}{8} + a\,n,$$

before the back end's own compression, so the method pays off when a is well below 7/8 and the flags are themselves regular.

5.2 Results

Back end, kennedy.xls	Plain (bytes)	Record delta	Activity map	Decode plain / activity
gzip -9	209,721	42,971	35,558	2,148 / about 1,400 µs
LZ4 -12	324,136	43,653	39,997	739 / about 1,000 µs
Snappy	422,640	129,763	69,635	966 / 1,030 µs
LZFSE	136,282	54,844	41,227	1,064 / 1,098 µs

The activity map gives the smallest output of any transform tried on record data, but its per-byte branch makes reconstruction slower than the fastest decoders. It is No-Pain only with gzip; with LZ4, Snappy and LZFSE it is a size option with a decode cost. In front of HCM it made output 5% to 35% larger, because HCM's own record contexts model the same structure, though it cut HCM decode time on kennedy.xls from 0.71 to 0.27 s. Of the five encodings of the flag stream tested, one bit per byte, packed, was smallest.

6 HCM: additions to an lpaq1-based context-mixing compressor

On the Canterbury corpus (2,810,784 bytes), HCM v3.4 max gave 350,326 bytes and decoded in 3.4 s, against 350,542 bytes and 11.5 s for ZPAQ -m5 on the same machine. HCM's core is not new: it follows lpaq1 [3] (Section 3). The contribution is a set of additions that took the same engine from 414,602 to 353,494 bytes in fast mode without slowing decoding.

6.1 Model

For each bit, n context models give probabilities p_j, which are mixed in the logistic domain with weights chosen by a small selector context s:

$$\widehat{p} = \sigma\left(\sum_{j=1}^{n} w_{s,j}\ \mathrm{st}\ \left(p_j\right)\right), \qquad \mathrm{st}\ (p) = \ln\frac{p}{1-p}, \qquad w_{s,j} \leftarrow w_{s,j} + \eta\ (y - \widehat{p})\ \ \mathrm{st}\ \left(p_j\right),$$

where y is the coded bit. An adaptive probability map refines the result, and an arithmetic coder [5] spends $-\log_2$ of the probability given to the actual bit.

6.2 Additions

Version	Addition	Canterbury, fast (bytes)
v3	Baseline	414,602
v3.1	Binary-only record swap: three context slots become row-above contexts when a record length is found	361,093
v3.2	Mixer selected by position in the word (8,192 weight sets)	358,884
v3.3	Line-position context; retuned refinement weight and match rate	353,494
hcmx (max, best-of)	Ternary pointer, record contexts (HCM-Bin)	347,696 (max)
v3.4	Match recovery after one wrong byte; 16-bit numeric context; raw store	350,326 (max)

HCM-Bin, the record-context variant, took kennedy.xls to 16,797 bytes against 16,822 for ZPAQ -m5, decoding in 0.79 s against 6.35 s.

6.3 The ternary pointer and its control

The ternary pointer first codes a flag, "is the next byte the predicted candidate c?", and on a miss codes the byte with c excluded. For a calibrated model this costs exactly the same as coding the byte directly:

$$-\log_2 P(c)\ \text{ on a hit}, \qquad -\log_2 (1 - P(c)) - \log_2 \frac{P(x)}{1 - P(c)} = -\log_2 P(x)\ \text{ on a miss}.$$

A control experiment confirmed this. Coding the flag with the model's own probability changed the total by 81 bytes; all of the measured 2,192-byte gain came from the refinement mixer re-estimating P(c), a form of secondary estimation. An earlier claim that a hit saves a byte's full cost was withdrawn.

6.4 Reference points

HCM v3.3 with a GPL English dictionary from cmix cut text output by 5.4%, but paq8px and cmix remain 9% to 12% smaller than the best HCM result. HCM sits in speed tier 4 (Section 2.4); its aim is the best ratio at a decode speed several times faster than ZPAQ's strongest mode, not the best ratio overall.

7 The Harrington Exchange

On packed 12-bit Gaussian noise (524,288 bytes), the Harrington Exchange followed by zstd -19 produced 324,507 bytes, within about 1% of the order-0 entropy bound of about 322,000 bytes and only 1,354 bytes (0.4%) larger than paq8px -8, the strongest context-mixing compressor tested. It decoded in **5.1 ms against paq8px's 189 s: about 37,000 times faster** †. It is a reversible pre-sort and split for biased random data packed at any bit width.

† paq8px and ZPAQ decode times are single wall-clock runs including program start-up; Exchange times are medians of 15 in-process runs. Start-up takes well under one second, so it cannot account for a gap of this size.

7.1 Method

The input is read as chunks of S bits. Let π rank the 2^S chunk values by frequency, and let τ order the target patterns from most to least desirable, here by the number of zeros in the leading J bits. The swap, a pre-sort, renames each chunk, and the juggle splits it:

$$v' = \tau(\pi\,(\,v\,)), \qquad \text{stream}_1 \leftarrow \left\lfloor v'/2^{S-J} \right\rfloor, \qquad \text{stream}_2 \leftarrow v' \bmod 2^{S-J}\ (\text{stored reversed}),$$

for a split point J with 0 < J < S. The map is a permutation, so it is exactly invertible; its table is stored with the Harrington shrinking header, which encodes a 256-entry permutation in 225 bytes. The back end then codes two streams whose statistics are far more skewed than the interleaved original. A full sweep of all 276 (S, J) pairs at zstd -19 was made on each file.

7.2 Results

Data (original bytes)	Exchange	Split only	zstd -22	Blosc2 best	ZPAQ -m5	paq8px -8
12-bit noise (524,288)	324,507	368,065	430,667	360,045	326,799	323,153
NIST biased bits (125,000)	19,722	22,375	22,191	21,640	18,986	18,064
NIST biased bytes (1,000,000)	321,128	321,353	327,396	321,626	292,664	290,460
5-bit skewed (524,289)	346,370	346,363	401,574	401,446	373,175	351,007
x-ray packed to 12 bits (525,000)	358,593	342,496	452,104	347,041	317,968	292,973

On the four biased files the Exchange was the smallest of the practical codecs tested (zstd, Brotli, 7-Zip, Blosc2, OpenZL), and on the 5-bit file it was smaller than paq8px. An ablation shows the gain needs both steps: renaming alone is worse than doing nothing, and full Exchange is 11.5% to 11.9% smaller than split alone on the biased sources. On real images and EEG, whose neighbouring samples are numerically close, renaming hurts and plain byte shuffle or Blosc2 wins.

The compiled decoder rebuilds symbols bit by bit, so it is 5 to 12 times slower than plain zstd; on 12-bit noise it is still about 3.8 times faster than xz -9e (5.1 against 19.2 ms) and smaller. Related methods are bitshuffle and byte shuffle [13], move-to-front, slot-plus-extra-bits codes and canonical Huffman ordering; the claim is the combination of a frequency-ranked permutation onto prefix-weight targets with a free split at arbitrary widths.

7.3 Biased random data

Biased random data is where the Exchange is strongest, and it is a real class of data: hardware random-number sources before whitening, analogue-to-digital converter output, sensor noise and quantised measurement residuals. Such data has no repeated strings for a dictionary coder to find, so LZ-family compressors gain little, while its symbol frequencies are uneven enough that an entropy coder can gain a lot.

Why the pre-sort helps. Split an S-bit symbol X into its leading J bits A and its trailing bits B. Coding the two streams separately costs at least

$$H(A) + H(B) = H(X) + I(A;B),$$

so the split alone loses the mutual information I(A;B) between the parts. The pre-sort maps the most frequent values onto targets whose leading bits are mostly zero. That concentrates the skew of the distribution in A, which the back end then codes almost at its entropy, and leaves B close to uniform and nearly independent of A, which keeps I(A;B) small. On data whose values are numerically smooth, such as images and EEG, the pre-sort destroys the closeness of neighbouring values and the effect reverses.

Results on biased sources. Sizes in bytes; decode times for the Exchange are medians of 15 in-process runs.

Source (original bytes)	Exchange	Order-0 bound	paq8px -8	Best practical rival	Exchange decode	ZPAQ -m5 decode	paq8px -8 decode
12-bit Gaussian noise (524,288)	324,507	about 322,000	323,153	360,045 (Blosc2)	**5.1 ms**	3,284 ms	**189,253 ms**
NIST biased bits (125,000)	19,722	17,688	18,064	21,640 (Blosc2)	**1.49 ms**	444 ms	**27,381 ms**
NIST biased bytes (1,000,000)	321,128	—	290,460	321,626 (Blosc2)	**7.4 ms**	4,002 ms	**269,710 ms**
5-bit skewed symbols (524,289)	346,370	about 340,300	351,007	388,457 (Brotli 11)	**11.3 ms**	—	—

The Exchange was the smallest practical result on all four sources. Against the context-mixing compressors it trades size for speed: on 12-bit noise it is 0.4% larger than paq8px and about 37,000 times faster to decode; on the NIST biased bits and bytes paq8px is 8% to 10% smaller but about 18,000 to 36,000 times slower; on the 5-bit source the Exchange is smaller than paq8px outright. ZPAQ -m5 decodes 300 to 640 times slower than the Exchange.

Choosing S and J. A full sweep of all 276 (S, J) pairs showed the optimum is broad: on the biased bits every pair from 13/21 to 16/24 was within 0.5% of the best. A cheap search over a few pairs is therefore enough in practice.

8 No-Pain or Big Gain tuning of stock encoders

7z-hi made every file in Silesia and Canterbury smaller and none slower to decode: Silesia went from 48,757,480 to about 47,333,100 bytes, and its total decode from 3,483 to 3,300 ms. Every result in this section is read by the unmodified decoder; only the encoder changes.

8.1 Method

For each file f, the encoder searches a set C of valid stock-format settings, including filters configured by the record detector, and keeps

$$c^{*}(f) = \arg\min_{c \in C} S_c(f) \quad \text{subject to} \quad \mathrm{dec}_{\mathrm{stock}}(\mathrm{enc}_c(f)) = f, \quad \tilde{T}_c(f) \le \tilde{T}_0(f),$$

falling back to the baseline when no candidate qualifies. Gains that come only from choosing a third party's existing level are reported but not credited; gains that depend on the record detector, such as a delta distance or a row width, are.

8.2 Results

Tool	Corpus	Baseline (bytes)	Tuned (bytes)	Decode baseline → tuned
7z-hi	Silesia (211,938,580)	48,757,480	about 47,333,100	3,483 → 3,300 ms
7z-hi	Canterbury (2,810,784)	494,902	464,853	64 → 59 ms
xz-hi	Canterbury	493,080	462,832	33.0 → 28.8 ms
lzma-hi	Silesia	about 48,752,900	about 48,191,700	2,939 → 2,825 ms
bzip2-hi	Canterbury	about 542,700	about 531,900	59.7 → 55.4 ms
zstd-hi, level 1	Canterbury	686,780	678,908	2.291 → 2.287 ms
Brotli-hi, quality 11	Canterbury	490,662	486,199	7.367 → 7.206 ms
PDF row filters	139 pdf.js test PDFs	81,681,005 (qpdf)	81,173,302	11.35 → 11.20 s render
ORC, detected columns	x-ray, zlib	6,168,696	4,695,458	46.1 → 25.6 ms read
Parquet, detected columns	x-ray, zstd	8,133,025	5,295,112	137 → 102 ms read

8.3 Notes

The largest single gains come from the record detector choosing a delta distance for 7-Zip and xz (kennedy.xls 51,868 → 23,656 bytes with xz, decoding 2.2 times faster, 6.48 → 2.90 ms) and a PNG Up row width for PDF streams. zstd and Brotli gain little, under 1.1%; their decode times are corpus totals of per-file medians of three runs, in process. PDF render times are the total of medians of three poppler renders of the first three pages of each file. For PDFs, the first three pages of every file rendered pixel-identical in poppler; compatibility with other readers was not tested. For Parquet and ORC, reshaping unknown binary data into the detected record width won on four of seven files and lost on ptt5, sum and osdb, where the plain layout is kept.

9 Harrington Sets: content-grouped parallel streams

The encoder makes one pass to label every window, a second to build the streams, and then compresses the streams in parallel. The decoder reads the map, decodes the streams in parallel and writes each window back to its original position.

9.1 Windows and features

The input of n bytes is cut into W = ⌈n / w⌉ windows of w = 65,536 bytes (the last may be shorter). For window i, let c_b be the count of byte value b and p_b = c_b / |w_i|. Its byte entropy in bits per byte is

$$H_i = -\sum_{b=0}^{255} p_b \log_2 p_b, \qquad 0 \le H_i \le 8.$$

Each window is described by a feature vector of seven shares, each in [0, 1]: control bytes (0–31 except tab, line feed and carriage return), letters, digits, spaces and line breaks, high bytes (128–255), zero bytes, and H_i / 8.

$$\mathbf{f}_i = \left(r_i^{\mathrm{ctl}},\ r_i^{\mathrm{alpha}},\ r_i^{\mathrm{digit}},\ r_i^{\mathrm{space}},\ r_i^{\mathrm{high}},\ r_i^{\mathrm{zero}},\ H_i/8 \right) \in [\,0,1\,]^7$$

Computing all features needs one 256-entry histogram per window, so the classification pass costs O(n) time and O(1) memory per window. The features overlap by design: zero bytes are counted both as control bytes and in their own share, and tab is counted with spaces.

9.2 Exclusion

A window is excluded to the raw set, label 255, when

$$H_i > \tau, \qquad \tau = 7.99 \text{ bits per byte}.$$

Such windows are random or already compressed; under an order-0 view the possible gain is at most 8 − H_i bits per byte, under 0.13% at the threshold, while modelling costs full time. A higher-order model can find structure that order-0 entropy misses, so the threshold is set close to 8. They are stored unchanged. A first trial with τ = 7.95 excluded 1.9 MB of mozilla that HCM could still shrink slightly, so the threshold was raised.

9.3 Grouping into sets

The remaining windows are clustered by k-means with k ≤ 3, minimising the within-set squared distance

$$\min_{\{\boldsymbol{\mu}_s\},\ \ell} \sum_{i:\ \ell_i \neq 255} \left\| \mathbf{f}_i - \boldsymbol{\mu}_{\ell_i} \right\|_2^2, \qquad \ell_i \in \{0,1,2\}.$$

The labels ℓ_i are then adjusted in three deterministic steps. The number of sets is k = 3 when at least 48 windows are modelled, 2 when at least 16 are, and 1 otherwise; centres start from the first modelled window and are added by farthest-point selection, followed by 20 k-means iterations. Any set smaller than 1 MB is then folded into the largest set. Finally, in two passes, a run of fewer than m = 8 consecutive windows (512 KB) takes the label of the run before it; the first run of the file keeps its label. Only windows of at least 4 KB can be excluded as raw. Without smoothing, mozilla split into 257 runs; with it, into 8.

9.4 Streams and blocks

The windows of each set are concatenated in file order into one stream. A stream longer than B bytes is cut into ⌈|S_s| / B⌉ blocks, with B = 16 MB by default (HCM_BLOCK_MB). Every block is compressed by a fresh copy of the unmodified single-core compressor, one thread per block, up to the number of cores.

9.5 Layered map and container

The label sequence is stored as runs (label, length). With R runs the map costs

$$M = 5R \text{ bytes.}$$

The exclusion is layered: the first level separates modelled windows from raw windows, and the second assigns modelled windows to sets. The container (magic "HCMS") holds the header, the map, the per-block lengths and the compressed blocks; raw windows are stored as-is. Because each window keeps its file-order position within its set, the decoder rebuilds the file by walking the map once.

9.6 Analysis

The size cost of Harrington Sets comes almost entirely from models that start empty, not from the map. On samba the map held 3 runs, 15 bytes, against a total cost of 5,416 bytes.

Cold-start cost. Splitting an input of n bytes into blocks of at most B bytes costs, as a heuristic approximation rather than a derived bound,

$$\Delta S(B) \approx \left(\left\lceil \tfrac{n}{B} \right\rceil - 1 \right) L(B) + 5R,$$

where L(B) is the extra cost of learning the statistics again in one block, which grows slowly with B. On dickens with 2 MB blocks the measured cost was 75,885 bytes over four extra cold starts, so L is about 19 KB per 2 MB text block. Grouping like windows lowers the cost in a second way: when a stream holds one kind of data, its model does not spend capacity relearning after each switch between kinds. That is the effect seen on samba, where equal blocks cost 43,151 bytes and sets cost 5,416.

Speed-up. Let the blocks take times $t_1, \ldots, t_K$ on one core, with K blocks, and let a serial part take a fraction f of the single-core time (classification, I/O, map). On P cores the decode time T_P and speed-up satisfy

$$T_P \geq \max\left(\max_k t_k,\ \frac{1}{P}\sum_{k=1}^{K} t_k \right), \qquad \frac{T_1}{T_P} \leq \min\left(P,\ K,\ \frac{\sum_k t_k}{\max_k t_k},\ \frac{1}{f + (1-f)/P} \right).$$

The last term is Amdahl's law [14]. Two practical limits follow. First, a file cannot use more cores than it has blocks: samba produced 3 blocks at B = 16 MB, so using 40 cores would need blocks of about 540 KB, with a larger cold-start cost. Second, unequal blocks waste cores: mozilla's 5 blocks on 2 cores ran in 3 rounds, and the slowest round set the time.

A conservative projection. Treating the samba result as if it were limited only by a serial fraction gives f = 2/1.73 − 1 ≈ 0.16. This overstates f, because it also absorbs load imbalance. Under that assumption the decode speed-up would be about 2.7 times on 4 cores, 3.8 times on 8 and 5.7 times on 40, with a ceiling of about 6.4. These figures are projections, not measurements: the model ignores the extra cold-start cost of smaller blocks, memory-bandwidth limits and scheduling effects, any of which can lower the real speed-up.

Memory. Each block running at once holds its own model, about 32 MB in fast mode and 64 MB in max mode, so peak memory grows as

$$M_{\mathrm{peak}} \approx \ \min\ (\ P, K\ ) \cdot M_{\mathrm{model}},$$

about 1.3 GB for 40 concurrent blocks in fast mode. Memory, not cores, may be the binding limit on large servers.

9.7 Results

On the heterogeneous file samba, sets were smaller and faster than equal blocks; on the four more uniform files, they were 0.05% to 0.8% larger than equal blocks, and decoded between 8% slower (mozilla) and 15% faster (reymont). All runs used HCM fast mode on a 2-core cloud machine, one run each, wall clock, with exact round trips. Medium files used 2 MB blocks; samba and mozilla used 16 MB.

File (bytes)	Mode	Size (bytes)	Ratio	Compress (s)	Decode (s)
samba (21,606,400)	single stream	3,400,668	6.354	62.9	16.8
	equal blocks	3,443,819	6.274	59.2	13.9
	Harrington Sets	3,406,084	6.343	42.5	9.7
mozilla (51,220,480)	single stream	13,355,236	3.835	180.8	47.0
	equal blocks	13,412,925	3.819	130.2	32.7
	Harrington Sets	13,419,759	3.817	135.9	35.4
dickens (10,192,446)	single stream	2,224,460	4.582	33.1	8.3
	equal blocks	2,300,345	4.431	20.4	5.6
	Harrington Sets	2,319,439	4.394	19.9	5.5
x-ray (8,474,240)	single stream	3,666,372	2.311	34.2	8.6
	equal blocks	3,683,815	2.300	17.8	5.0
	Harrington Sets	3,710,615	2.284	17.5	4.5
reymont (6,627,202)	single stream	1,093,130	6.063	18.7	5.1
	equal blocks	1,121,748	5.908	12.7	3.3
	Harrington Sets	1,129,732	5.866	10.2	2.8

The medium files were drawn at random (seed 1793303107) from Canterbury and Silesia files up to 11 MB; the other two drawn, alice29.txt and ptt5, fit in one block and gained nothing, with about 30 bytes of map added. Equal-block parallelism on lpaq1-h, measured as a reference, decoded dickens, x-ray and reymont 1.6 to 1.9 times faster for 1.4% to 5.5% more size.

The first version of the method, with τ = 7.95 and no smoothing, split mozilla into 257 runs and excluded 1.9 MB that could still be compressed; it was 0.5% larger than the tuned version. Only two cores were available; no measurement beyond two cores has been made. Each configuration was run once, so differences of about 10% or less in time, such as sets against equal blocks on dickens, x-ray and reymont, should be treated as unconfirmed until repeated.

10 Summary of results

The strongest results are on tabular and biased data; on ordinary text and executables most methods fall back to the stock tool. Sizes are compressed bytes.

Method	Class	Data	Baseline	Result	Decode	Grade
Record delta + zstd -22	Mutated	kennedy.xls	64,818	26,401	2.6× faster	No-Pain
Combary-variant activity map + gzip -9	Original method, Mutated pipeline	kennedy.xls	209,721	35,558	about 1.5× faster	No-Pain
Record delta + xz -9e (built-in filter)	Modified	kennedy.xls	51,868	23,656	2.2× faster	No-Pain
Record delta + zstd -19	Mutated	x-ray	5,129,823	4,508,494	18% faster	No-Pain
7z-hi	Modified	Silesia	48,757,480	about 47,333,100	5% faster	No-Pain
xz-hi	Modified	Canterbury	493,080	462,832	13% faster	No-Pain
PDF row filters	Modified	139 PDFs	81,681,005 (qpdf)	81,173,302	11.35 → 11.20 s render	No-Pain
ORC detected columns, zlib	Mutated	x-ray	6,168,696	4,695,458	1.8× faster read	No-Pain
HCM v3.4 max vs ZPAQ -m5	Mutated	Canterbury	350,542	350,326	3.4× faster	No-Pain against ZPAQ
HCM-Bin vs ZPAQ -m5	Mutated	kennedy.xls	16,822	16,797	8× faster	No-Pain against ZPAQ
Harrington Exchange + zstd vs xz -9e	Original	12-bit noise	403,956	324,507	3.8× faster	No-Pain against xz
Harrington Exchange + zstd vs paq8px -8	Original	12-bit noise	323,153	324,507 (+0.4%)	**5.1 ms vs 189 s, about 37,000× faster** †	BGS-3
Harrington Sets vs single stream	Original	samba	3,400,668	3,406,084 (+0.16%)	1.73× faster	BGS-1

† See the note in Section 7.

11 Negative results and limits

Most ideas tested did not work, and nearly every failure either broke the byte alignment that compressors rely on or added a decoding pass slower than the compressor it was meant to help.

Idea	Result
HaBiTS, a headerless binary-to-ternary conversion, as a modelling alphabet	Canterbury 552,136 vs 437,409 bytes, 26% worse
Binary-to-ternary conversion in front of HCM	54% to 67% worse
Bit-plane splitting in front of zstd, LZ4 and gzip	No gain on any file
8-bit Harrington Exchange outside record data	Larger on 9 of 11 Canterbury files
Nibble permutation alone (Quad Exchange)	Never helped; a permutation preserves every repeated string
Activity map in front of HCM	5% to 35% larger
Text/binary sections inside one HCM model (v3.5)	samba unchanged, mozilla 3,263 bytes smaller
LZ4 with any separate inverse pass	No win: LZ4 decodes near memory speed
Ternary pointer flag on its own	81 bytes on Canterbury; the gain was secondary estimation

Re-verification. A random re-test on 28 October 2026 reproduced the sizes of the sampled wins, allowing for two cases measured with a different library version. Some timing claims did not hold: record delta on x-ray with zstd -19 decoded slower until the prefix-sum inverse was written; lzma-hi on sao and two of three Brotli-hi samples were ties; the Exchange prototype decoder was slower than its baseline.

Limits of the evidence. All results are the author's measurements on 2-core cloud machines, and many timings are single runs. Some Silesia figures were derived from normalised sizes and are marked "about". The corpora are standard but small; the PDF results were checked with one reader. Harrington Sets has not been measured beyond two cores. No formal literature or patent search has been made, so claims of originality mean only that no prior match was found.

12 Discussion

Three lessons run through the results. First, structure that a general compressor cannot see, such as a fixed record width, is worth more than any tuning of the compressor itself. Second, a transform must preserve the alignment the back end relies on; the failures in Section 11 broke it. Third, a transform is only free if its inverse is faster than what it saves, which is why the vectorised inverses mattered more than the transforms themselves.

Where the methods apply. Record delta, the activity map and detected columns apply to fixed-record data: spreadsheets, instrument and sensor dumps, medical images, database exports. The Exchange applies to biased or odd-width numeric data, such as ADC samples and entropy-source output, and not to smooth signals. Encoder tuning applies wherever files are written once and read many times. Harrington Sets applies to slow, stateful compressors on large mixed files.

Beyond compression. Harrington Sets separates three steps that need no compression: format-free sorting of data by measured content, exclusion of material not worth processing, and parallel per-group workers with an exact map back. That pattern could serve backup and storage systems that skip encrypted or compressed regions, hardware compressors that need a fixed-cost classifier, AI data pipelines that sort mixed corpora before

training, and any slow adaptive process limited to one core. These uses are hypotheses; none has been tested. Entropy scanning on its own is established in forensics, so the claim is the combination.

Independent testing. Andrew Goodwin of Sydney, Australia got the author's software working on his own systems and found the results viable. He ran HCM and the other compressors on three benchmark sets of his own choosing: the Canterbury corpus, a 12.6 MB Silesia sample and an 18 MB synthetic set, on a single 2.1 GHz Xeon core, and his measurements were consistent with the compression ratios reported here. His feedback shaped later work: points from his review led to HCM v3.4 (match recovery, a 16-bit numeric context, raw storage of incompressible files and a stricter check on old files); his observation that HCM's core is close to lpaq1, paq8 and ZPAQ led to the credits in Section 3; and his note that context-mixing compressors are slow and memory-hungry for many uses motivated the multi-core work in Section 9. On his synthetic set, lpaq1-h was the smallest of the compressors he tested (8,113,745 bytes against 8,202,053 for ZPAQ -m5). A run on a many-core server is the next step needed to measure Harrington Sets beyond two cores.

13 Conclusion and future work

Cheap, measured structure detection, combined with fast inverses and the No-Pain and Big Gain rules, improved a wide range of standard compressors on tabular and biased data, and Harrington Sets gave a single-core context-mixing compressor a 1.73-fold decode speed-up on two cores for 0.16% more size on a mixed file. On ordinary text and executables the gains are small or absent, and the paper reports where each method fails.

Future work, in order of expected value:

1. Measure Harrington Sets on 4 to 40 cores, with block size chosen per core count, and record peak memory.
2. Fuse the record-delta inverse into the LZ4 and zstd copy loops, so the fastest tier can win too.
3. Make the activity-map rebuild branch-free with SIMD masks.
4. Test the HCM refinement estimator as a plain mixer input or SSE stage, without the pointer flag.
5. Choose sets by trial compression instead of byte statistics, and warm each set's model with a shared sample.
6. Check PDF output with several readers, and run a formal literature and patent search.

References

1. M. V. Mahoney, "Adaptive weighing of context models for lossless data compression," Florida Institute of Technology, Tech. Rep. CS-2005-16, 2005.
2. M. V. Mahoney et al., paq8 and paq8px series of context-mixing compressors, 2003–2026.
3. M. V. Mahoney, lpaq1, 2007, mattmahoney.net/dc.

4. B. Knoll, cmix, byronknoll.com/cmix.html.

5. I. H. Witten, R. M. Neal and J. G. Cleary, "Arithmetic coding for data compression," Communications of the ACM, 30(6), 520–540, 1987.

6. Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, 2021.

7. M. V. Mahoney, "The ZPAQ open standard format for highly compressed data," specification, 2009–2016.

8. S. Deorowicz, Silesia compression corpus, Silesian University of Technology, 2003.

9. R. Arnold and T. Bell, "A corpus for the evaluation of lossless compression algorithms," Proc. Data Compression Conference, 1997.

10. Meta, OpenZL format-aware compression framework, 2025.

11. Apache Software Foundation, Apache Parquet and Apache ORC file formats.

12. J. Liu et al., "Logzip: Extracting hidden structures via iterative clustering for log compression," Proc. IEEE/ACM ASE, 2019.

13. K. Masui et al., "A compression scheme for radio data in high performance computing," Astronomy and Computing, 12, 181–190, 2015.

14. G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS Conference Proceedings, 30, 483–485, 1967.

15. W. H. Hsu and A. E. Zwarico, "Automatic synthesis of compression techniques for heterogeneous files," Software: Practice and Experience, 25(10), 1097–1116, 1995.

16. J. Amit and O. Shalev (IBM), "Separation of data chunks into multiple streams for compression," US Patent 8,782,019 B2, 2014; and J. Amit et al. (IBM), "Real-time classification of data into data compression domains," US Patent 9,792,350 B2, 2017.

Credits, license and data availability

HCM's core design follows Matt Mahoney's lpaq1, with parts from paq8, lpaq and ZPAQ, the Witten–Neal–Cleary arithmetic coder and Arm Optimized Routines (exp and log tables). lpaq1-h and zpaq-hi are modified versions of Mahoney's software and keep their original licenses, as do the stock tools tuned in Section 8. The author's own methods and code are released under the MIT License, copyright 2026 Michael Hugh Harrington, and are offered free for use in the original tools.

All source code, test scripts, per-file results and notes are in the public repository github.com/CombaryMaker/Harringtons_Insights. A longer account with full per-file tables is the book

Harrington's Insights: Improvements, Concepts, Strategies, Successes and Failures in Data Compression (2026), in the same repository.